Lecture 2: Parallel Hardware & Software

Institute of Computer Science & Information Technology,
Faculty of Management & Computer Sciences,
The University of Agriculture, Peshawar, Pakistan.

Parallel Hardware and Software

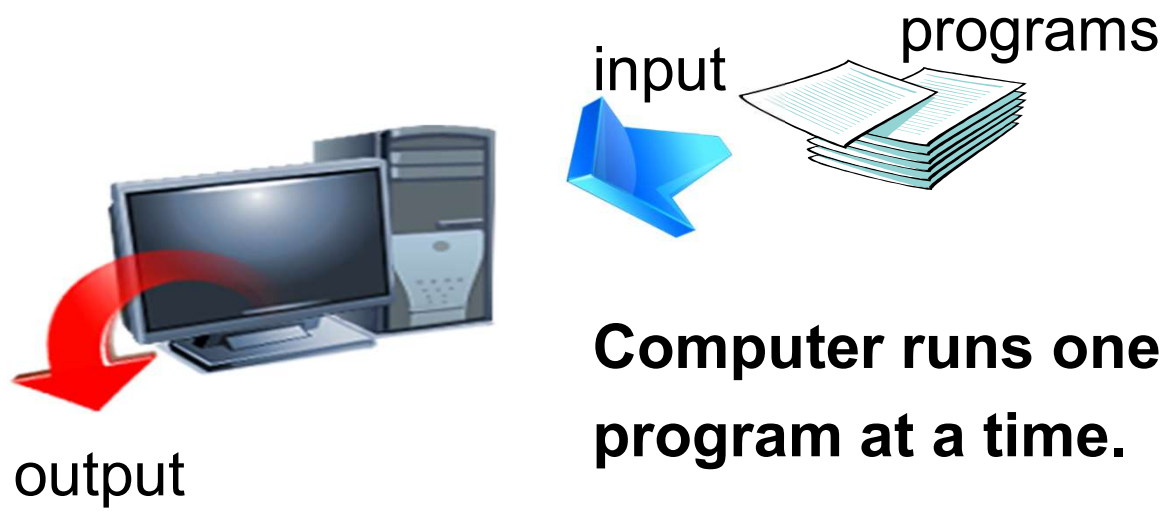
- To understand and write efficient parallel programs:
  - Knowledge of underlying parallel hardware
  - Knowledge of Parallel Software

Hardware Architectures

- It refers to the identification of system's physical components and their interrelationships.
- It is set of rules and methods that describe the functionality, organization and implementation of computer.
- It is also concerned with balancing the performance, efficiency, cost and reliability of a computer system.
  - Performance: Computer architects strive to design systems that deliver optimal performance for the intended tasks.
  - Efficiency: Efficiency refers to achieving maximum output with minimal resources.
  - Cost: Architects must balance performance and features against manufacturing costs, component prices, and overall system affordability.
  - Reliability: Reliability is essential for ensuring that computer systems operate correctly under normal conditions and continue to function in the face of faults or errors.

Serial hardware and software

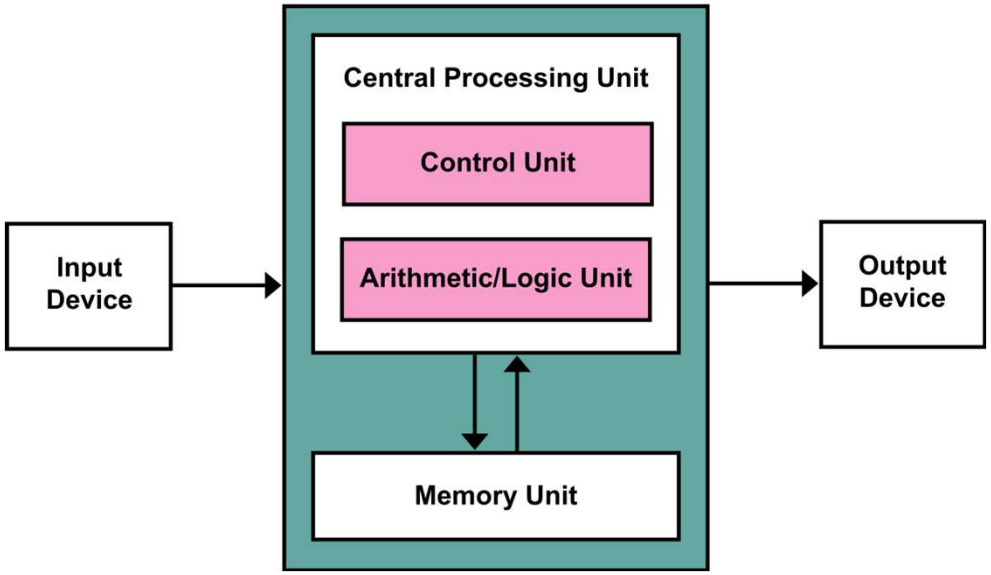
- Parallel systems evolved from serial systems.
  - Serial systems: Hardware and software designed to run a single job at a time.
  - Understanding serial systems helps explain modern parallelism.

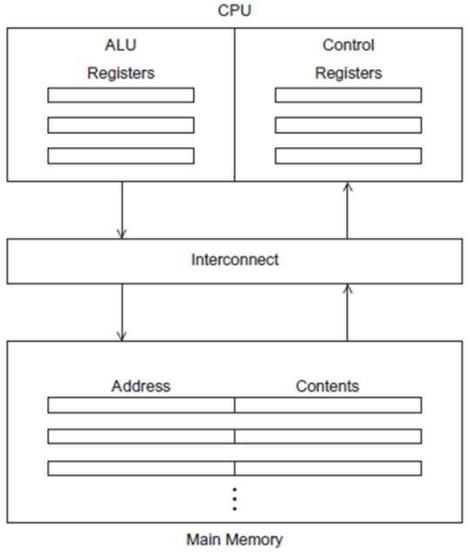


Computer runs one program at a time.

Von Neumann Architecture

- Stored Program Concept: Instructions and data reside in the same memory.
- Sequential Execution: Instructions are fetched and executed one after another.
- Bottleneck: Single bus can cause performance limits (Von Neumann Bottleneck).





CPU Architecture Types: Scalar, Vector, Pipelined, Superscalar, and VLIW

Scalar Architecture

- Executes **one instruction per clock cycle** on a single data element.
- **Characteristics:**
  - Simple design, single pipeline.
- **Example:** Early microprocessors like Intel 8086.
- Instruction Stream → [Instruction] → [ALU] → Result
  (One at a time)

Vector Architecture

- Operates on **multiple data elements** in a single instruction.
- **Characteristics:**
  - Designed for data-parallel tasks (e.g., scientific computing).
  - Uses vector registers to hold arrays of data.
- **Example:** Cray-1 supercomputer, modern GPUs.
- **Diagram:**
- Instruction: VADD V1, V2

```
  V1:      [a1    a2     a3     a4]
+ V2:      [b1    b2     b3     b4]
-----------------------------------
  V3:      [c1    c2     c3     c4]
```

- All elements computed in parallel

Instruction Level Parallelism (ILP)

- Attempts to improve processor performance by having multiple processor components or **functional units** simultaneously executing instructions.
- **Pipelining** - functional units are arranged in stages.

- **Multiple issue (Super Scalar)** - multiple instructions can be simultaneously initiated.

Pipelining

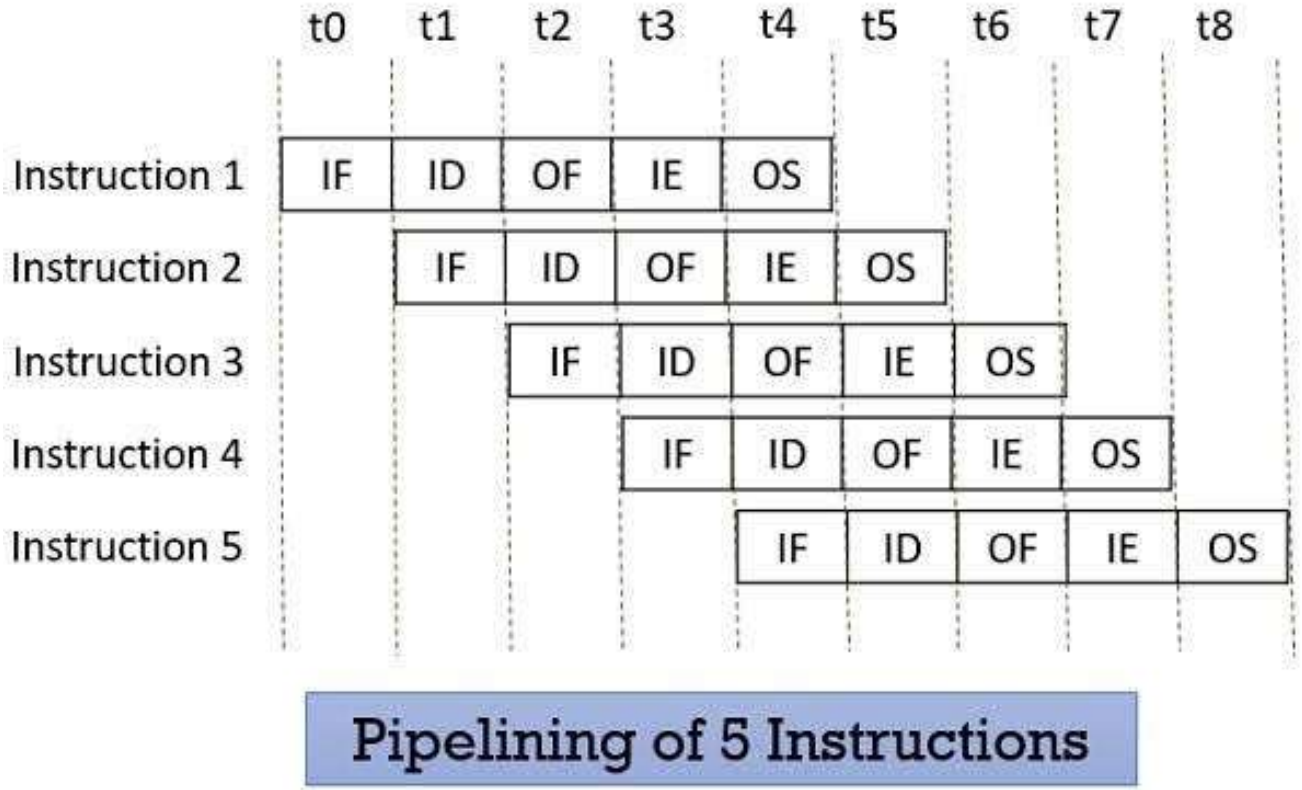


Pipelined Architecture

- Breaks instruction execution into **stages** (fetch, decode, execute, etc.), allowing multiple instructions to be overlapped.
- **Characteristics:**
  - Increases instruction throughput.
  - Each stage works on a different instruction simultaneously.
- **Example:** MIPS R2000, ARM Cortex-M.
- **Diagram:**
- Cycle →1 2 3 4 5 6
  Instr 1: [IF] [ID] [EX] [WB]
  Instr 2: [IF] [ID] [EX] [WB]
  Instr 3: [IF] [ID] [EX] [WB]

  (Staggered execution, like an assembly line)

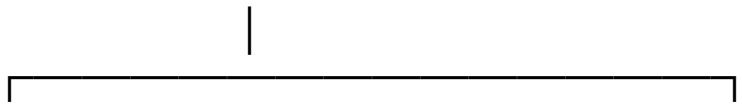
Pipeline Arch. Example



Pipelining of 5 Instructions

Superscalar Architecture

- Can issue and execute **multiple independent instructions per cycle** using multiple pipelines.
- **Characteristics:**
  - Instruction-level parallelism (ILP).
  - Requires multiple ALUs, load/store units, and complex scheduling logic.
- **Example:** Intel Pentium (two-way superscalar), ARM Cortex-A series.
- **Diagram:**
- Instruction Decoder



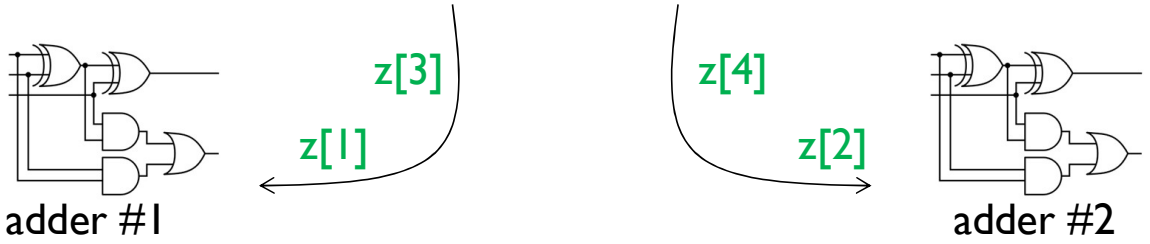
Pipeline A: [ADD] → [ALU1]
Pipeline B: [LOAD] → [ALU2]

(Both executed in same cycle if independent)

Cont.

- Such processors replicate functional units and try to simultaneously execute different instructions in a program.

```
for (i = 0; i < 1000; i++)
    z[i] = x[i] + y[i];
```



Cont.

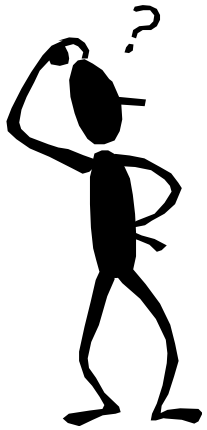
- **static** multiple issue (multiple pipeline) - functional units are scheduled at compile time.

- **dynamic** multiple issue (multiple pipeline) – functional units are scheduled at run-time.

superscalar

Speculation (1)

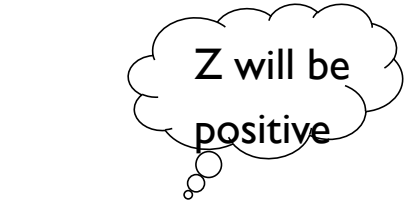
- In order to make use of multiple issue (multiple pipeline), the system must find instructions that can be executed simultaneously.



  - In speculation, the compiler or the processor makes a guess about an instruction, and then executes the instruction on the basis of the guess.

Speculation (2)

```
z = x + y ;
i f ( z > 0)
    w = x ;
e l s e
    w = y ;
```





If the system speculates incorrectly,
it must go back and recalculate w = y.

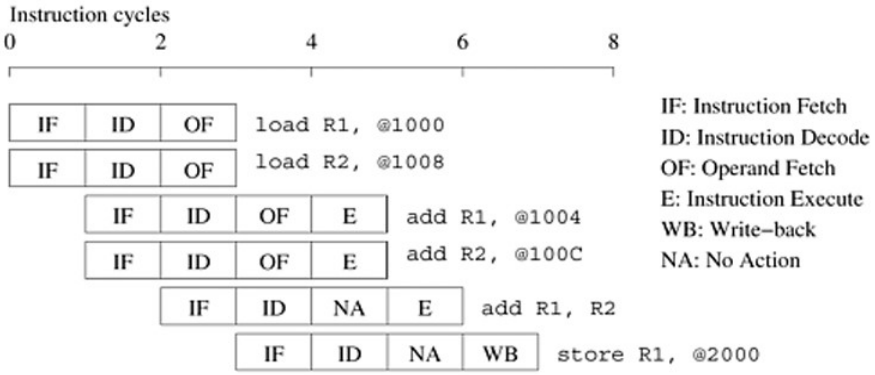
Super Scalar (Example)

Figure 2.1. Example of a two-way superscalar execution of instructions.

```
1. load R1, @1000      1. load R1, @1000      1. load R1, @1000
2. load R2, @1008      2. add R1, @1004       2. add R1, @1004
3. add R1, @1004       3. add R1, @1008       3. load R2, @1008
4. add R2, @100C       4. add R1, @100C       4. add R2, @100C
5. add R1, R2          5. store R1, @2000     5. add R1, R2
6. store R1, @2000                            6. store R1, @2000
```

(i) (ii) (iii)

(a) Three different code fragments for adding a list of four numbers.



(b) Execution schedule for code fragment (i) above.

Very Long Instruction Word (VLIW)

- Compiler packs **multiple operations into a single long instruction word** for parallel execution.
- **Characteristics:**
  - Hardware is simpler (less dynamic scheduling).
  - Relies on compiler to extract parallelism.
- **Example:** Intel Itanium, DSP processors.
- **Diagram:**
- VLIW Instruction Word:
  [ADD R1,R2 | MUL R3,R4 | LOAD R5,@1000]

  → ALU1: ADD
  → ALU2: MUL
  → Load Unit: LOAD
  (All triggered in the same cycle)

Cont.

- Lets suppose we have d=ax+by+cz
- We can group the individual instructions like below and will take 5 cycles
- $Cycle\ 1$: $\begin{cases} Load\ a \\ Load\ x \end{cases}$
- $Cycle\ 2$: $\begin{cases} Load\ b \\ Load\ y \\ mul\ (a, x)\ \ AX \end{cases}$
- $Cycle\ 3$: $\begin{cases} Load\ c \\ Load\ z \\ mul\ (b, y)\ \ BX \end{cases}$
- $Cycle\ 4$: $\begin{cases} mul\ (c, z)\ \ CX \\ Add\ (AX, BX)\ \ AX \end{cases}$
- $Cycle\ 5$: $\begin{cases} Load\ (AX, CX) \end{cases}$

Comparison

Architecture	Parallelism	Who Handles Scheduling	Example
Scalar	1 instruction/cycle	Hardware (sequential)	Intel 8086
Vector	1 vector instr = many ops	Hardware	Cray-1, GPUs
Pipelined	Overlapped stages	Hardware	MIPS R2000
Superscalar	Multiple instr/cycle	Hardware (dynamic scheduling)	Intel Pentium
VLIW	Multiple instr/cycle	Compiler	Intel Itanium

Flynn's Taxonomy

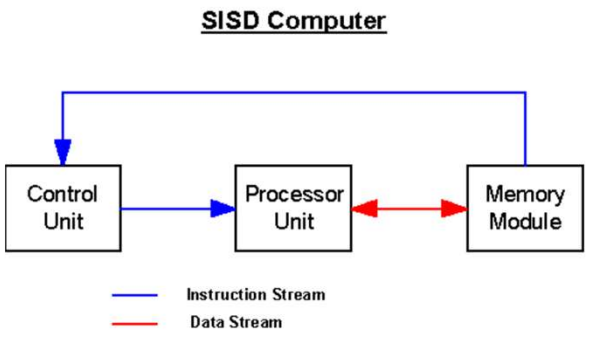
- Micheal J. Flynn (born May 20, 1934) is an American Professor at Stanford university.

- He proposed the Flynn's taxonomy, a method of classifying digital computer architectures, in 1966.

- Two types of Information flow into a processor:

1. Instruction Stream: The sequence of instructions from memory to the control unit.

2. Data Stream: The sequence of data from memory to the control unit.

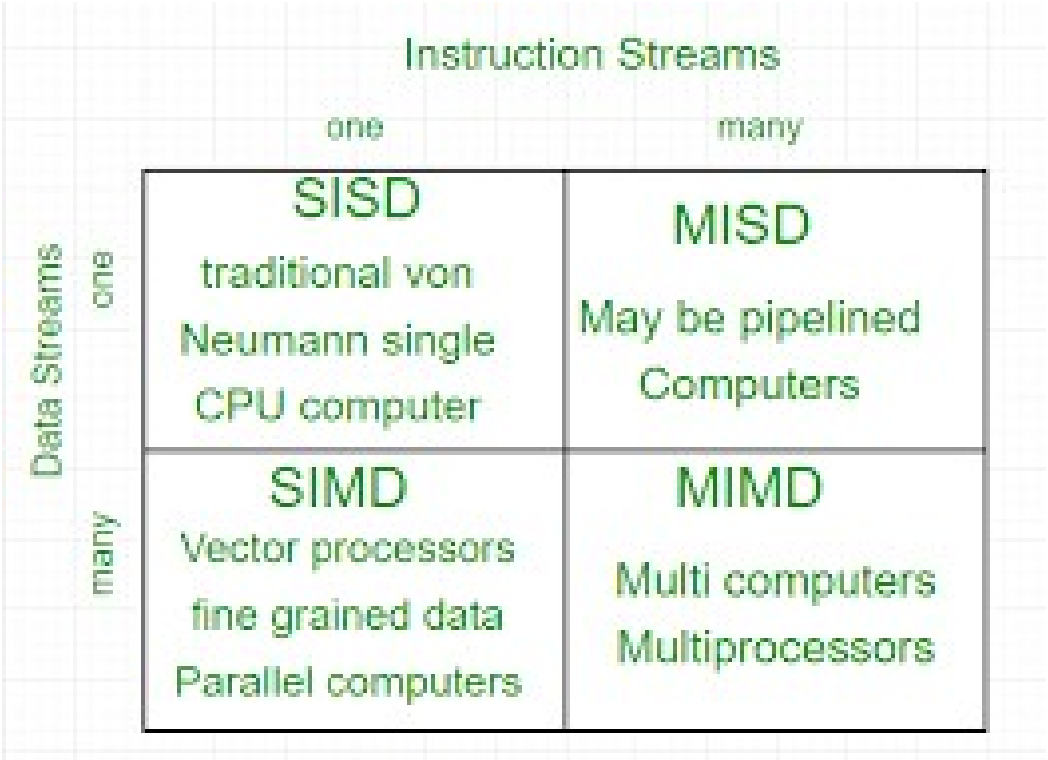


Flynn's Classification

- Number of concurrent instructions (or control) streams and data stream available in the architecture can be single or multiple.

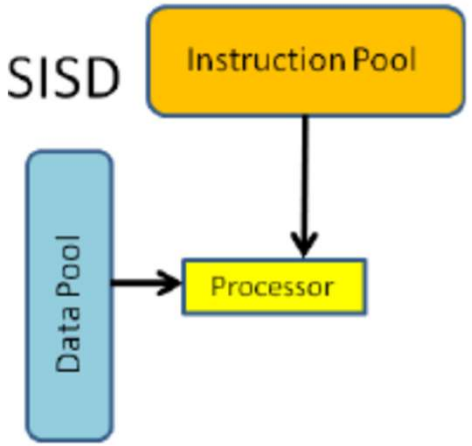
1. Single Instruction Stream, Single Data Stream (SISD)
2. Single Instruction Stream, Multiple Data Streams (SIMD)
3. Multiple Instruction Streams, Single Data Stream (SISD)
4. Multiple Instruction Streams, Multiple Data Streams (MIMD)

Flynn's Classification(1)



Single-Instruction Single Data (SISD)

- It is uni-processor machine which is capable of executing a single instruction, operating on single data stream.
- Machine instructions are processed in a sequential manner in which single control unit fetches single instruction from memory.
- The control unit then generate appropriate control signals to direct single processing element to operate on single data steam i.e., one operation at a time
- The speed of the processing element in the SISD model is limited by the rate at which the computer can transfer information internally.

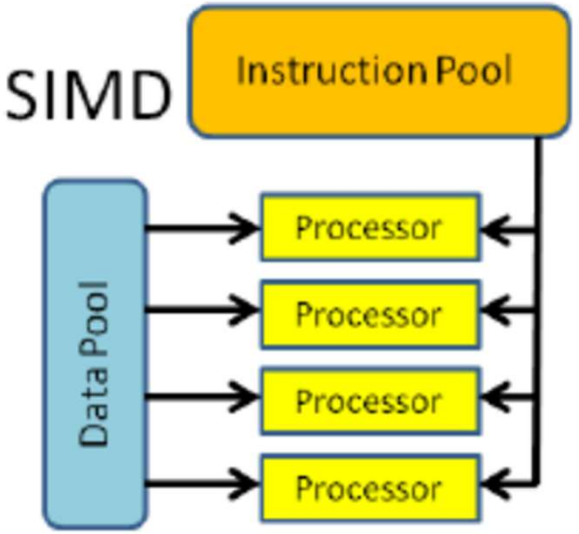


Single-Instruction Single Data (SISD)(1)

- It require less power
- There is no issue of complex communication protocol between multiple cores.
- The speed of SISD architecture is limited just like single-core processors.
- Dominant representative of SISD systems are IBM PC, Workstations

Single-Instruction Multiple Data (SIMD)

- It is a multiprocessor machine capable of executing the same instruction on all the CPUs but operating on different data streams.
- Machines based on an SIMD model are suited to scientific computing since they involve lots of vector and matrix operations.
- The information can be passed to all the Processing Elements (PE).
- Organized data elements of vectors can be divided into multiple sets and each PE can process one data set.

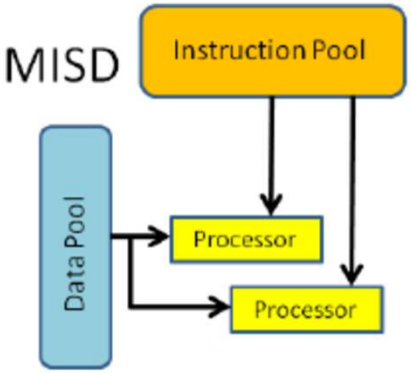


Single-Instruction Multiple Data (SIMD)(1)

- Instructions can be executed sequentially, such as by pipelining, or in parallel by multiple functional units.
- Throughput of the system can be increased by increasing the number of cores of the processor.
- Processing speed is higher than SISD architecture.
- There is complex communication between number of cores of processor.

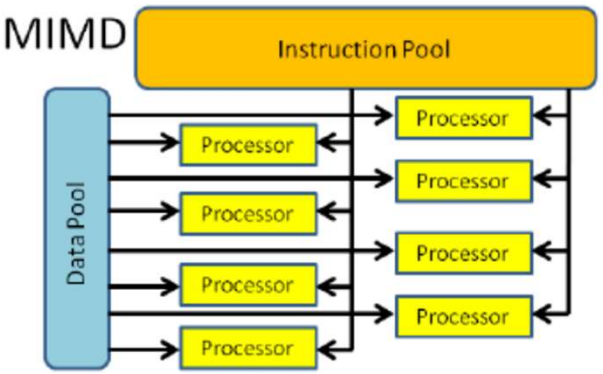
Multiple-Instruction Single Data (MISD)

- In this computing, microprocessor is capable of executing different instructions on PE but all of them operating on the one data stream.
- It has number of processing units performing different operations by executing different instruction on the same dataset e.g.
  - sin(x) + cos(x) + tan(x)
- Maybe used in fault-tolerant systems where data is replicated and processed by multiple redundant processors using different algorithms.
- This model is not useful in general for most of the applications.



Multiple-Instruction Multiple Data (MIMD)

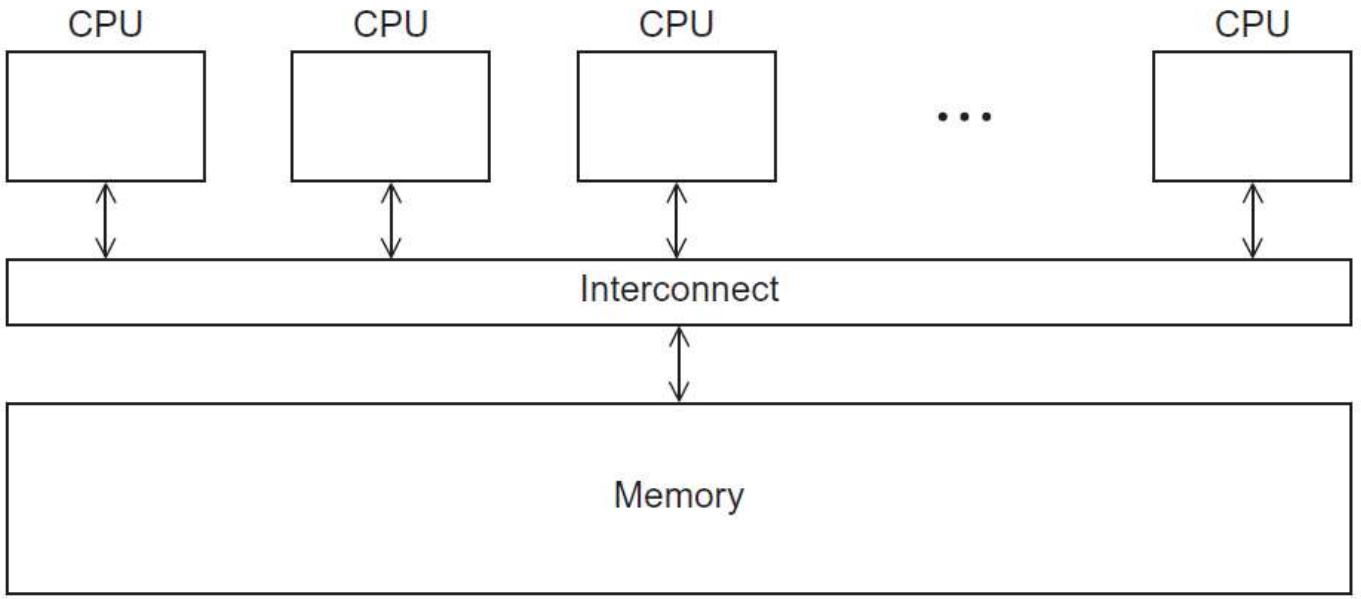
- This is model used for multiprocessor, which is capable of executing multiple instructions on multiple data sets.
- Each PE in the MIMD has separate instructions and data streams; therefore machines built using this model are capable to any kind of applications.
- Unlike SIMD and MISD machines, PEs in MIMD machines work asynchronously.
- MIMD architectures include multi-core processors, and distributed systems using either one shared memory space of distributed memory space.



Shared Memory System

- A collection of autonomous processors is connected to a memory system via an interconnection network.
- Each processor can access each memory location.
- The processors usually communicate implicitly by accessing shared data structures.
- Most widely available shared memory systems use one or more multicore processors.
  - (multiple CPU's or cores on a single chip)

Shared Memory System

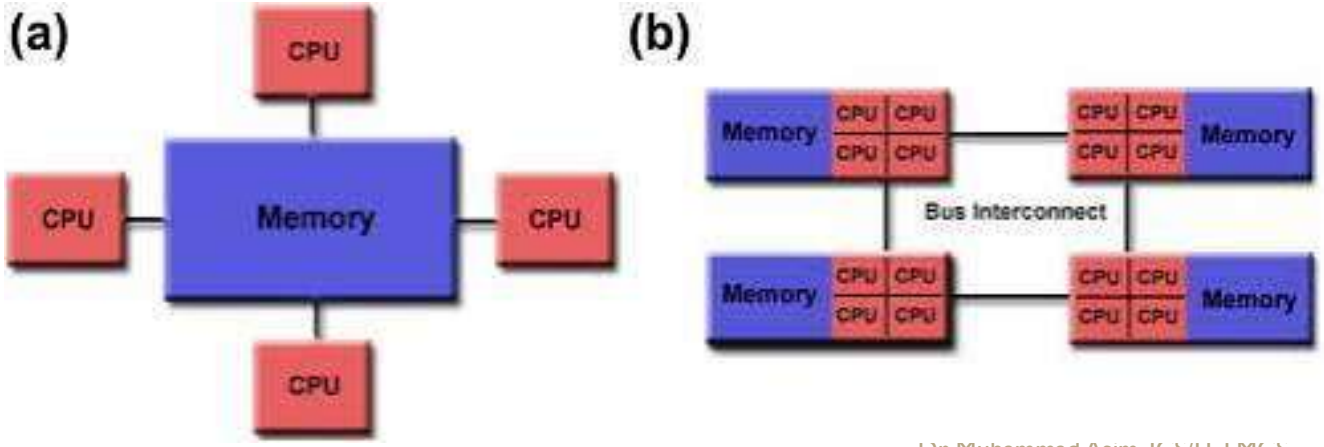


UMA vs NUMA multicore shared memory systems

- In shared memory systems following different approaches may followed:

1. ***Uniform Memory Access (UMA):*** All the processors share the physical memory uniformly (e.g., access time is same etc.)

2. ***Non-Uniform Memory Access (NUMA):*** memory access time depends on the memory location relative to a processor; A memory location a core is directly connected to can be accessed faster than a memory location that must be accessed through another chip.

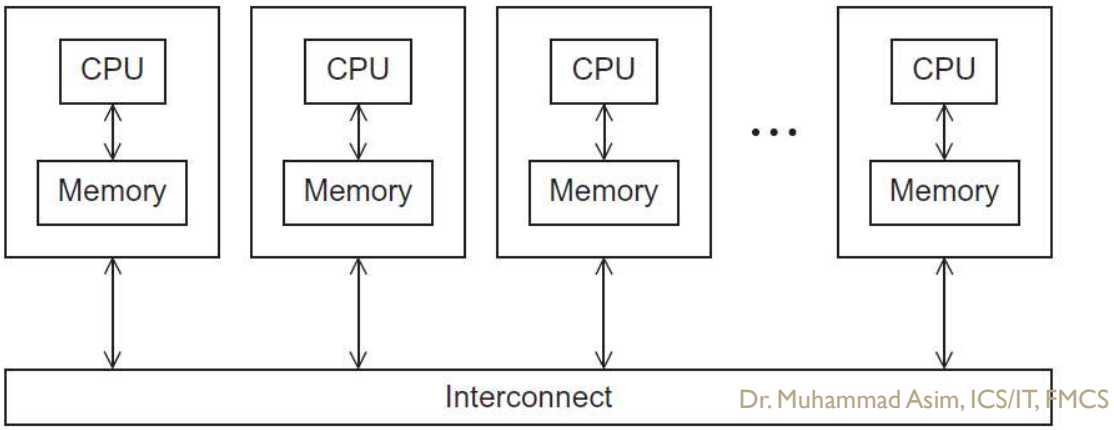


Distributed Memory System

- Clusters (most popular)
  - A collection of commodity systems.
  - Connected by a commodity interconnection network.

- Nodes of a cluster are individual computations units joined by a communication network.

a.k.a. hybrid systems



Advantage of Shared Memory

- Shared memory system is relatively easy to program since
  - all processor share a single view of data and
  - the communication between processors can be as fast as memory accesses to a same location.

Challenge in Shared Memory

- The issue with shared memory systems is that many CPUs need fast access to memory and will likely cache memory, which have two complications:

1. **Access Time Degradation:** When several processors try to access same memory location it causes contention
2. **Lack of Data Coherence:** Whenever one cache is updated with information that may be used by other processors, the change needs to be reflected to the processors, otherwise the different processor will be working with incoherent data.

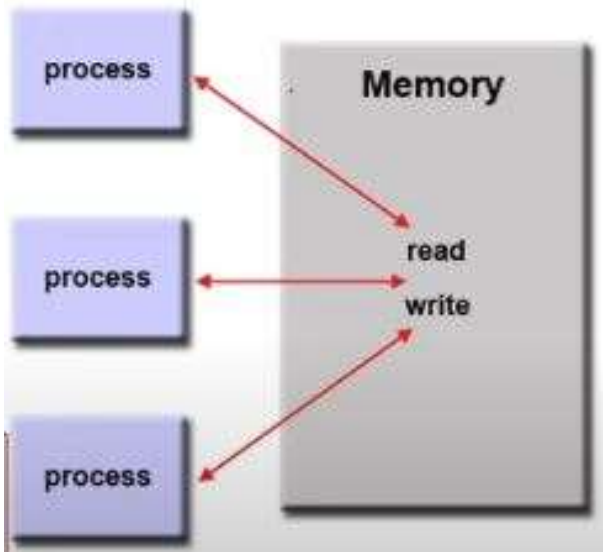
Challenge in Shared Memory (1)

- Various mechanisms can be used to aid the access time degradation:

1. Locks: Two values of lock can be possible, either 0 or 1. A process want to be in the critical section first check the lock variable.

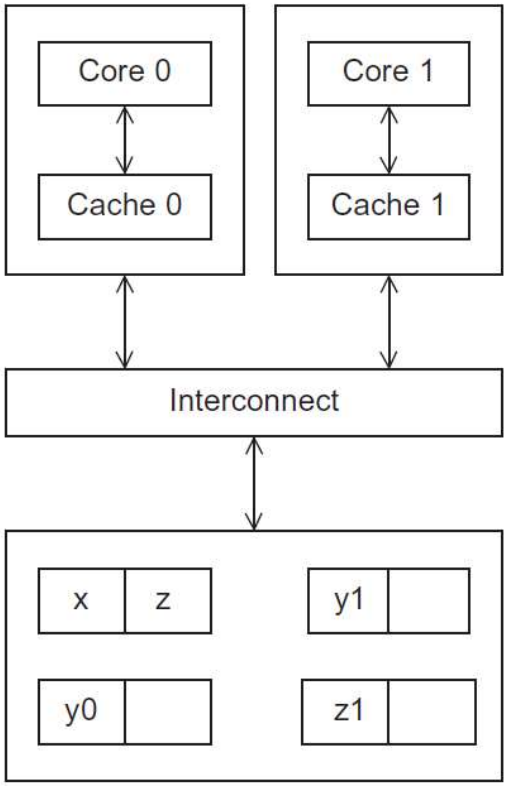
2. Semaphores used for the synchronization of concurrent processes.

3. Mutex is a locking mechanism used to synchronize access to a resources. Only one task can acquire mutex.



Cache coherence

- Programmers have no control over caches and when they get updated.



A shared memory system with two cores and two caches

Cache coherence

y0 privately owned by Core 0

y1 and z1 privately owned by Core 1

x = 2; /* shared variable */

Time	Core 0	Core 1
0	`y0 = x;`	`y1 = 3*x;`
1	`x = 7;`	Statement(s) not involving x
2	Statement(s) not involving x	`z1 = 4*x;`

y0 eventually ends up = 2

y1 eventually ends up = 6

z1 = ???

Snooping Cache Coherence

- The cores share a bus .
- Any signal transmitted on the bus can be "seen" by all cores connected to the bus.
- When core 0 updates the copy of x stored in its cache it also broadcasts this information across the bus.
- If core 1 is "snooping" the bus, it will see that x has been updated and it can mark its copy of x as invalid.

Directory Based Cache Coherence

- Uses a data structure called a directory that stores the status of each cache line.
- When a variable is updated, the directory is consulted, and the cache controllers of the cores that have that variable's cache line in their caches are invalidated.

Interconnection networks

- Affects performance of both distributed and shared memory systems.

- Two categories:
  - Shared memory interconnects
  - Distributed memory interconnects

Shared memory interconnects

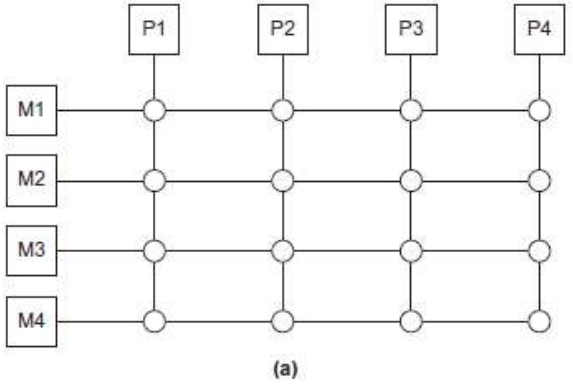
- Bus interconnect
  - A collection of parallel communication wires together with some hardware that controls access to the bus.
  - Communication wires are shared by the devices that are connected to it.
  - As the number of devices connected to the bus increases, contention for use of the bus increases, and performance decreases.

Shared memory interconnects

- Switched interconnect
  - Uses switches to control the routing of data among the connected devices.
  - Crossbar –
    - Allows simultaneous communication among different devices.
    - Faster than buses.
    - But the cost of the switches and links is relatively high.

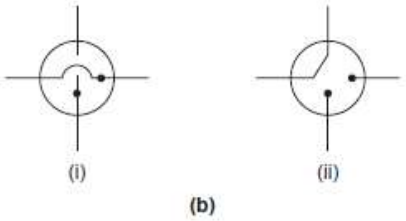
(a)

A crossbar switch connecting 4 processors (P_i) and 4 memory modules (M_j)

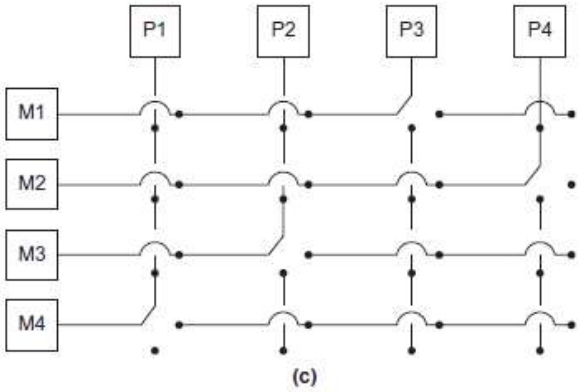


(b)

Configuration of internal switches in a crossbar



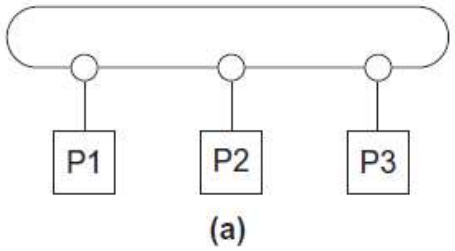
(c) Simultaneous memory accesses by the processors



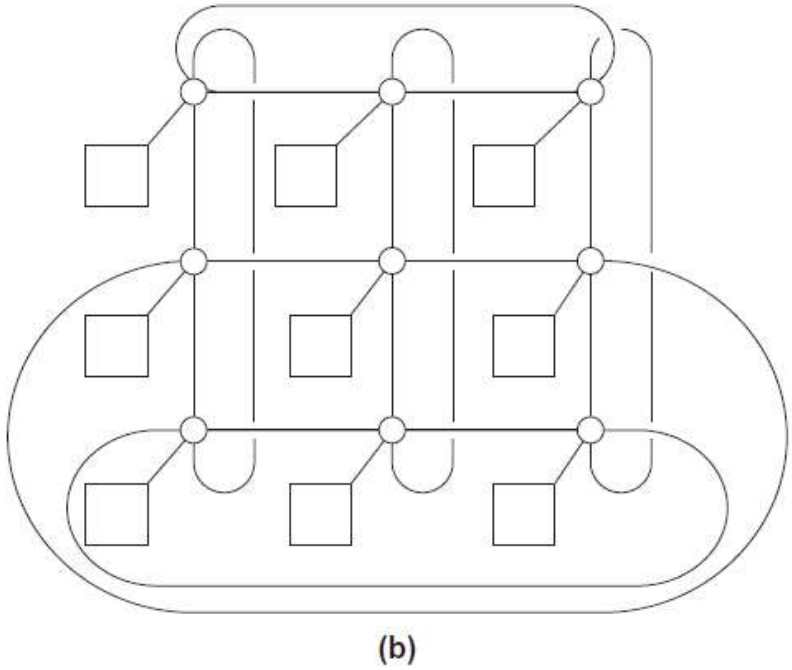
Distributed memory interconnects

- Two groups
  - Direct interconnect
    - Each switch is directly connected to a processor memory pair, and the switches are connected to each other.
  - Indirect interconnect
    - Switches may not be directly connected to a processor.

Direct interconnect



ring

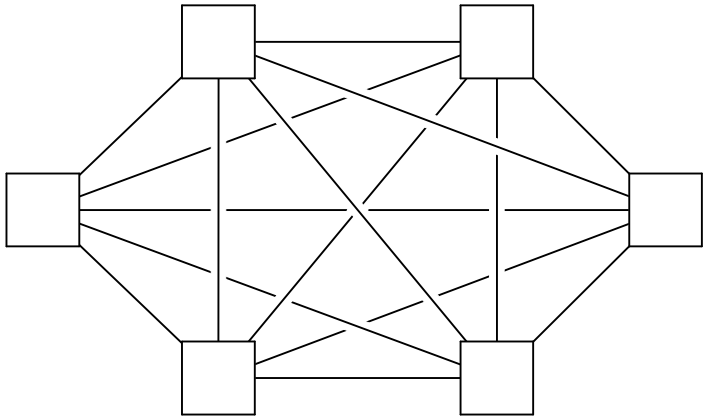


toroidal mesh

Fully connected network

- Each switch is directly connected to every other switch.

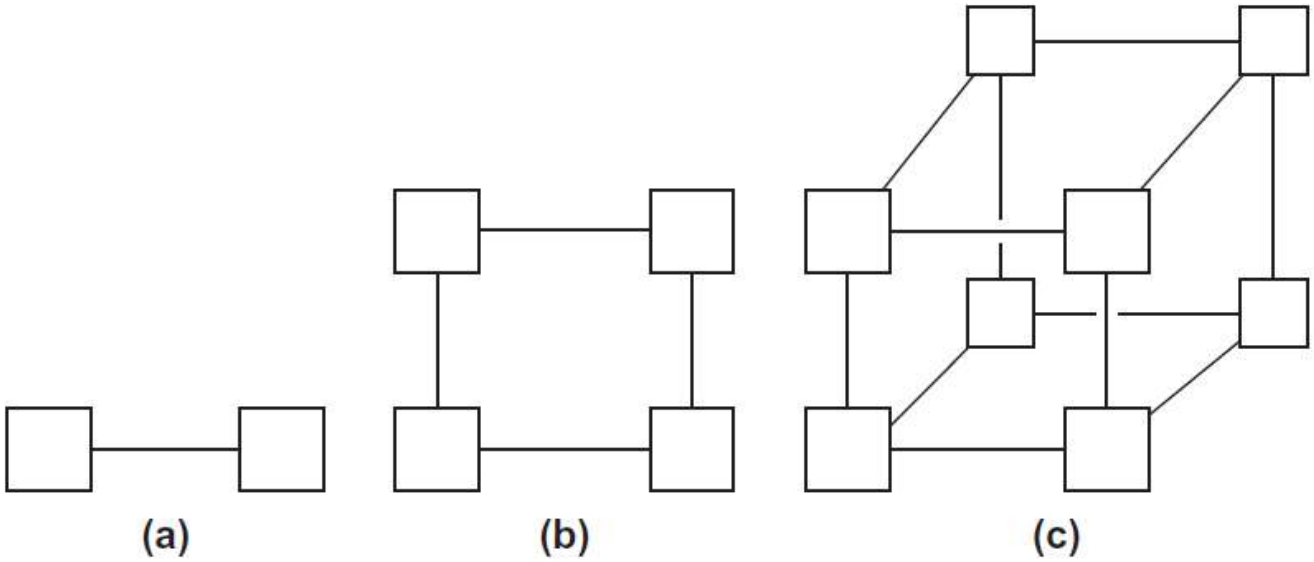
impractical



Hypercube

- Highly connected direct interconnect.
- Built inductively:
  - A one-dimensional hypercube is a fully-connected system with two processors.
  - A two-dimensional hypercube is built from two one-dimensional hypercubes by joining "corresponding" switches.
  - Similarly a three-dimensional hypercube is built from two two-dimensional hypercubes.

Hypercubes



one- two- three-dimensional

Indirect interconnects

- Simple examples of indirect networks:
  - Omega network
- Often shown with unidirectional links and a collection of processors, each of which has an outgoing and an incoming link, and a switching network.

A generic indirect network

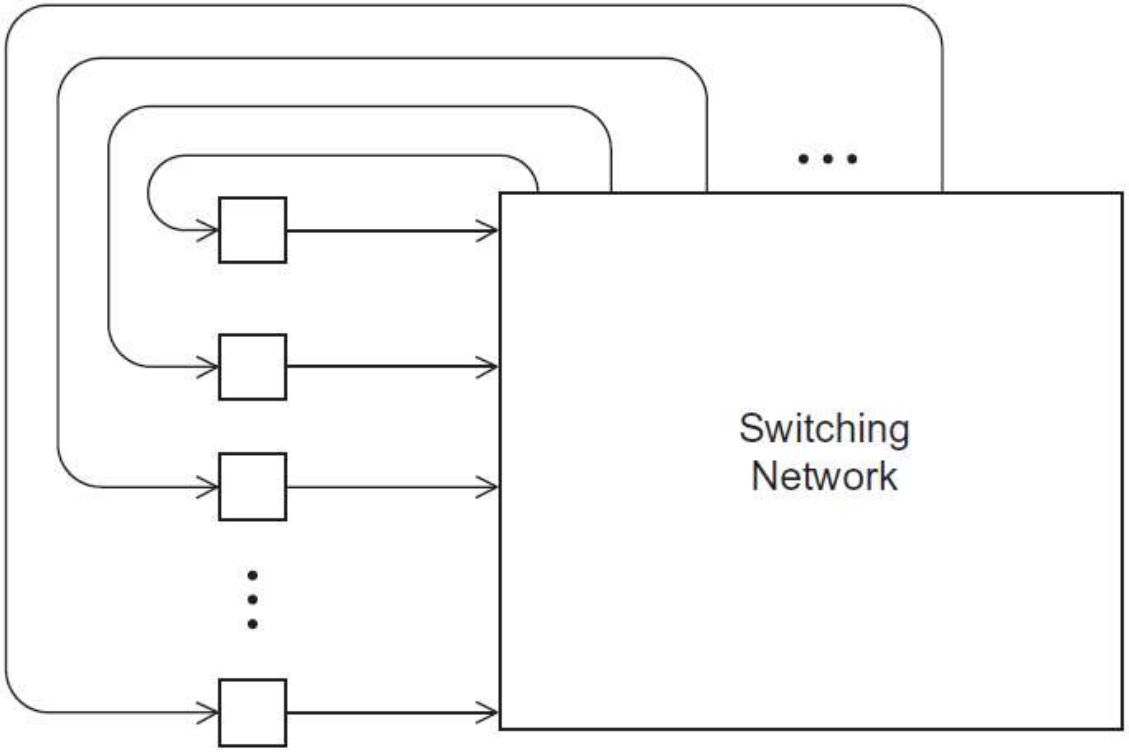
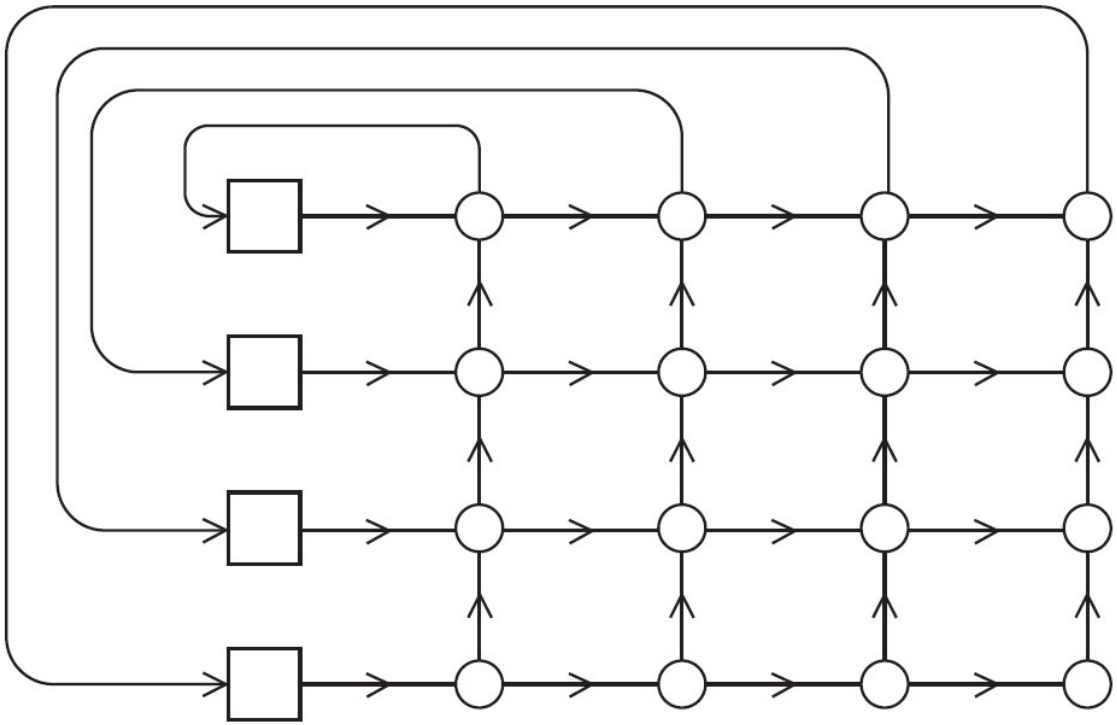
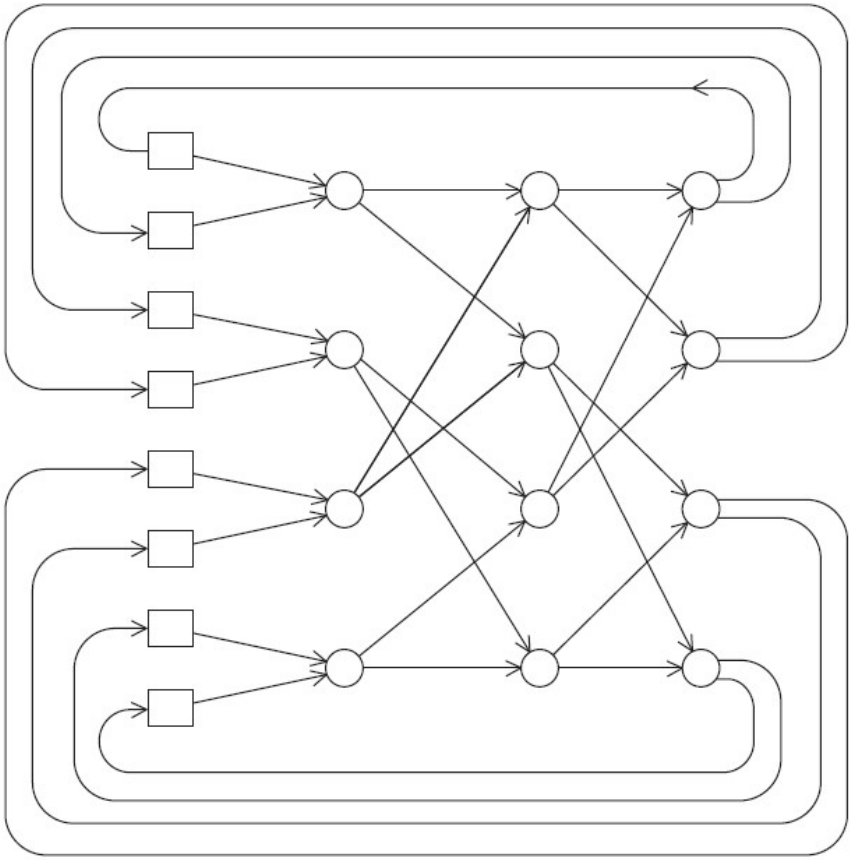


Figure 2.13

Crossbar interconnect for distributed memory



An omega network



More definitions

- Any time data is transmitted, we're interested in how long it will take for the data to reach its destination.
- Latency
  - The time that elapses between the source's beginning to transmit the data and the destination's starting to receive the first byte.
- Bandwidth
  - The rate at which the destination receives data after it has started to receive the first byte.

Message transmission time = l + n / b



latency (seconds)

length of message (bytes)

bandwidth (bytes per second)